REDIS & CACHING REVISION GUIDE

A Comprehensive Guide to In-Memory Data Structures, Caching Patterns, Persistence & Distributed Locks

A Concise, Visual, and Mechanism-Focused Interview & Revision Handbook

METADATA FIELD	SPECIFICATION
Subject:	In-Memory Computing & Distributed Systems
Target Audience:	SDE / Technical Interviews (Top Tech, FAANG, Product Firms)
Methodology:	Internal Mechanics • State Transitions • Verified Code Recipes • Trade-offs
Source File Consolidated:	Interview Concepts/redis.md (Commands, Strings, Lists, Sets, Hashes, TTL, Systemd)
Subject Coverage:	In-Memory Mechanics, RDB vs AOF, Data Structures, Cache-Aside, Eviction Policies, Distributed Locks
Key Architectural Focus:	Request Flows, Hit/Miss/Update Lifecycles, Cache Failure Mitigations & Cluster Hash Slots
Target Evaluation:	Technical Interview Placement Preparation - Core Engineering Track

CH	CHAPTER TITLE & CORE MECHANISM	CRITICAL INTERVIEW CONCEPTS & COMMAND RECIPES
1	Redis Fundamentals & In-Memory Architecture	RAM vs Disk, Single-Threaded Event Loop (ae), Sub-millisecond latency
2	Data Persistence: RDB Snapshots vs AOF Logs	Point-in-time fork copy-on-write RDB vs AOF append-only logs, fsync options
3	Core Data Structures & Essential Commands	Strings (SET/GET), Lists (LPUSH/RPOP), Sets (SADD), Hashes (HSET), ZSETs
4	Key Expiration, TTL & Eviction Policies	EXPIRE, SETEX, Active vs Passive expiration; allkeys-lru vs volatile-lfu
5	Caching Strategies & Request Flows	Cache-Aside (Lazy), Write-Through, Write-Behind (Write-Back), Refresh-Ahead
6	Atomic Operations & Distributed Locking	SETNX with PX and UUID, Lua script safe release, Redlock algorithm
7	Cache Failure Modes & Mitigation Patterns	Cache Penetration (Bloom Filters), Cache Avalanche (TTL Jitter), Cache Stampede
8	High Availability: Replication, Sentinel & Cluster	Leader-Follower replication, Sentinel quorum failover, Cluster 16384 hash slots

CHAPTER 1 - REDIS FUNDAMENTALS & IN-MEMORY ARCHITECTURE

Redis (Remote Dictionary Server) is an open-source, in-memory networked data store. Unlike traditional relational databases that read and write blocks to SSD/HDD storage, Redis keeps the entire active dataset in RAM, delivering sub-millisecond latencies and handling over 100,000 operations per second on a single node.

1.1 Why Is Redis So Fast? (The 3 Architectural Pillars)

1. **In-Memory RAM Storage:** RAM access latency is ~100 nanoseconds, compared to ~100 microseconds for NVMe SSDs (1,000x speed difference). Zero disk seek overhead during normal request execution.
2. **Non-Blocking I/O Multiplexing (epoll):** Redis uses a custom event library (ae.c) built on Linux epoll. A single thread monitors thousands of client sockets concurrently without multi-threading context switch penalties.
3. **Optimized C Data Structures:** Redis uses specialized internal encodings: SDS (Simple Dynamic Strings), ZipLists, QuickLists, SkipLists, and Dicts.

1.2 Systemd Service Management (from redis.md)

Code: Linux Systemd Redis Commands

```
# Managing the Redis Background Daemon
sudo systemctl start redis-server     # Start service
sudo systemctl stop redis-server      # Stop service
sudo systemctl restart redis-server   # Restart service
sudo systemctl status redis-server    # Inspect health and memory status
sudo systemctl enable redis-server    # Enable automatic start on boot

# Connecting via CLI:
redis-cli -h 127.0.0.1 -p 6379        # Interactive prompt
```

CHAPTER 2 - DATA PERSISTENCE: RDB SNAPSHOTS VS AOF LOGS

By default, RAM is volatile. Redis provides two complementary persistence mechanisms to reload data after reboots or crashes.

Dimension	RDB (Redis Database Snapshots)	AOF (Append-Only File)
Operating Mechanism	Point-in-time binary snapshot of all data written to dump.rdb.	Sequential append-only log recording every write command executed.
Data Loss Window	Moderate: All writes since the last snapshot are lost if crash occurs.	Minimal: At most 1 second of data lost (with default appendfsync everysec).
File Size & Compactness	Extremely compact binary format; ideal for off-site backups/S3.	Much larger file size; requires periodic background rewriting (BGREWRITEAOF).
Recovery Restart Speed	Fast: Directly loads binary snapshot directly into memory.	Slower: Must sequentially replay every logged write command.
Fork Overhead	Uses fork() with Copy-On-Write; can spike memory usage on large datasets.	Rewrite also uses fork(), but normal appending has negligible latency.

RECOMMENDED PERSISTENCE STRATEGY

Hybrid Persistence (Redis 4.0+): Enable both RDB and AOF (aof-use-rdb-preamble yes). During AOF rewrite, Redis writes an RDB binary snapshot as the file header and appends recent incremental writes as standard AOF logs. This delivers ultra-fast restarts with maximum durability!

CHAPTER 3 - CORE DATA STRUCTURES & ESSENTIAL COMMANDS

3.1 Master Command Reference (Consolidated from redis.md)

Data Structure	Key Commands	Time Complexity	Practical Use Case
Strings	SET k v, GET k, DEL k, EXISTS k, INCR k	O(1)	Caching session tokens, HTML fragments, distributed counters.
Lists / Arrays	LPUSH k v, RPUSH k v, LPOP k, RPOP k, LRANGE k 0 -1	O(1) push/pop, O(N) range	Message queues, recent activity feeds, event streams.
Sets (Unique)	SADD k v, SREM k v, SMEMBERS k, SISMEMBER k v	O(1) add/check	Unique visitor tracking, tag clouds, social graph mutual friends.
Hashes (Objects)	HSET k f v, HGET k f, HGETALL k, HDEL k f	O(1) field access	User profile objects, product attributes (avoids JSON serialization!).
Sorted Sets (ZSET)	ZADD k score v, ZRANGE k 0 -1, ZREVRANGE k 0 9	O(log N)	Gaming leaderboards, rate limiter sliding window timestamps.

Code: Redis CLI Command Recipes

```
# Practical Command Examples
SET user:1001:session "xyz890" EX 3600   # Store session with 1 hour expiration
HSET user:1001 name "Alice" role "Admin" # Store object fields
LPUSH task_queue "job_id_45"            # Push job to queue
RPOP task_queue                         # Worker pops job
ZADD leaderboard 1500 "PlayerA"         # Add to leaderboard with score 1500
```

CHAPTER 4 - KEY EXPIRATION, TTL & EVICTION POLICIES

4.1 Expiration Mechanics: Active vs Passive

Redis does not spawn a timer per key. Instead, expired keys are reclaimed via two combined mechanisms:
1. **Passive Expiration:** When a client reads a key (e.g., GET user:1), Redis checks its TTL; if expired, it deletes the key and returns nil.
2. **Active Expiration:** 10 times per second, Redis randomly samples 20 keys with TTLs. Any expired keys are evicted. If more than 25% of the sampled keys are expired, it repeats immediately to reclaim memory.

4.2 Memory Eviction Policies (When RAM Is Full)

Eviction Policy	Target Keys	Algorithm	Behavior on Write
noeviction (Default)	All keys	None	Returns error on new writes (OOM command rejected); read queries succeed.
allkeys-lru	**All keys**	Least Recently Used	Evicts the least recently accessed key across the entire database.
volatile-lru	Only keys with TTL	Least Recently Used	Evicts LRU keys among those that have an expiration set.
allkeys-lfu	**All keys**	Least Frequently Used	Evicts keys with the lowest access frequency counter (frequency-based).
volatile-ttl	Only keys with TTL	Shortest TTL	Evicts keys closest to their expiration time.

CHAPTER 5 - CACHING STRATEGIES & REQUEST FLOWS

Strategy	Read Workflow	Write Workflow	Pros & Trade-offs
Cache-Aside (Lazy Loading)	Read Cache. On miss: Read DB, populate Cache, return.	Write directly to DB, then **invalidate (delete)** key in Cache.	Resilient to cache node crashes; cache contains only requested data. Stale data window.
Write-Through	Read Cache. On miss: Cache fetches from DB.	App writes to Cache; Cache writes synchronously to DB before returning.	Zero stale data; high write latency (double write on every update).
Write-Behind (Write-Back)	Read Cache directly.	App writes to Cache only. Cache asynchronously flushes batch writes to DB.	Ultra-fast write throughput. Risk of permanent data loss if cache crashes before DB flush!
Refresh-Ahead	Read Cache directly.	Background thread predicts access and refreshes cache from DB before TTL expires.	Near zero cache misses for hot keys. Requires predictable access patterns.

CACHE-ASIDE WRITE INVARIANT: UPDATE VS DELETE

Should you update or delete the cache on DB writes?

ALWAYS **DELETE (Invalidate)** the cache key rather than updating it! Updating the cache creates race conditions: if Transaction 1 and Transaction 2 update DB in order T1 -> T2, but their cache updates arrive in order T2 -> T1, the cache is left permanently corrupted with stale T1 data. Deleting the key guarantees the next reader fetches authoritative DB data.

CHAPTER 6 - ATOMIC OPERATIONS & DISTRIBUTED LOCKING

6.1 Production Distributed Lock via SETNX

To acquire a mutual exclusion lock across distributed microservice instances without race conditions:
• **Atomic Acquisition:** SET lock_key unique_uuid NX PX 30000
- NX: Only set if key does NOT exist.
- PX 30000: Auto-expire in 30,000 ms (prevents permanent deadlock if holder crashes).
- unique_uuid: Random caller ID to prevent Node A from releasing Node B's lock!

6.2 Safe Lock Release via Atomic Lua Script

Code: Safe Distributed Lock Release (Lua)

```
-- Lua Script executed inside Redis (Atomically checks UUID before deletion!)
if redis.call("get", KEYS[1]) == ARGV[1] then
    return redis.call("del", KEYS[1])
else
    return 0  -- Lock expired or owned by another process!
end
```

CHAPTER 7 - CACHE FAILURE MODES & MITIGATION PATTERNS

Failure Scenario	Root Cause & Symptom	Catastrophic Impact	Architectural Mitigation
Cache Penetration	Queries for **non-existent keys** (e.g., ID = -999) bypass cache and hit DB on every request.	DB CPU spikes to 100% under malicious scan.	**1. Bloom Filter:** Pre-filter invalid keys. **2. Cache Nulls:** Store (key, null) with short 60s TTL.
Cache Avalanche	Massive batch of cached keys share the **exact same TTL and expire simultaneously**.	Sudden avalanche of thousands of requests hits DB simultaneously.	**TTL Jitter:** Add random offset to TTL (e.g., TTL = 3600 + random(0, 300) seconds).
Cache Stampede (Breakdown)	A single ultra-hot key expires while experiencing thousands of requests/sec.	Massive concurrent cache misses hammer DB with duplicate queries.	**Mutex Lock on Miss:** Only 1 thread acquires lock to query DB; all others wait.

CHAPTER 8 - HIGH AVAILABILITY: REPLICATION, SENTINEL & CLUSTER

Component	Architecture Topology	Failover Mechanism	Scaling Scope
Replication	Single Leader + Multiple Read Replicas (Asynchronous replication stream).	None built-in (manual promotion required if Leader dies).	Scales **Read throughput** only.
Redis Sentinel	Distributed monitoring cluster (typically 3 or 5 Sentinel processes).	**Automated Failover:** Monitors health via heartbeats; elects new Leader via Raft/Quorum.	High availability for single-shard setups.
Redis Cluster	Multi-master sharding across **16,384 Hash Slots** (CRC16 mod 16384).	Automatic slot rebalancing and node failover without proxies.	Scales **both Read and Write throughput** horizontally across multiple servers.

REDIS & CACHING INTERVIEW CHEAT-SHEET

- **Why Redis is fast:** In-memory RAM, single-threaded epoll event loop, zero context switches.
- **Persistence:** RDB (compact point-in-time snapshot) + AOF (durable write log); use hybrid persistence.
- **Cache-Aside:** Always *delete* cache on DB updates to avoid race condition corruption.
- **Distributed Lock:** `SET key uuid NX PX 30000` + Lua script check-and-delete.
- **Failure Defenses:** Penetration -> Bloom filter; Avalanche -> TTL jitter; Stampede -> Mutex lock on miss.